

## INF250 Assignment 2 – Chocolate Factory Image Analysis

Objective: Distinguish between Non-stops and M&Ms using image analysis techniques.

### Initial observation

My initial observation of the original image, image (1) in Figure 5, is that I easily can distinguish between Non-stops and M&Ms with my eyes. That is, the Non-Stops are more circular, have a darker shade of color, and they are “flatter”, whilst the M&Ms are more oval and cast more shadow from the light than the Non-Stops, meaning they stand more out from the background. I also observed that some Non-Stops are broken in half, while all M&Ms are intact.

### Pre-Processing

#### Grayscale conversion and cropping

I am converting the original image to grayscale when it is read in to the .ipynb file. See image (2) in Figure 5. This is to simplify the analysis of the image, because the image will go from having three color channels to one, reducing the data size. Then I crop the image, such that only the chocolates are viewed, essentially removing unnecessary distractions for the analysis, such as the ruler in the bottom.

```
# Loading the image as grayscale
img = io.imread(filepath, as_gray = True)

# Cropping the image
crop_margins = ((300, 300), (600, 600))
resized_image = crop(img, crop_margins)
```

Figure 1: Grayscale and cropping

### Thresholding

I then used the Otsu method for thresholding to again convert the image, now to a binary version. This results in image (3) in Figure 5, an image containing only two pixel values/gray scales. In which separates the chocolates from the background. Binarizing an image can also remove some noise in the image.

```
# Computing a Otsu threshold value
threshold_value = threshold_otsu(resized_image)
print(f"Threshold value: {threshold_value}")

# Manually selecting a threshold value and creating a binary version of the image
thres = 0.54
```

Figure 2: Thresholding

### Closing

In the original image we can see that the chocolates, especially the ones colored yellow reflect light. After the preceding preprocessing, this reflection is shown as black spots in the otherwise white chocolates, as seen in image (3) in Figure 5. Therefore, I have applied a closing operation to the image. The closing operation removes such black spots, so they are not mistakenly classified as background during further analysis. The result can be seen in image (4) in Figure 5.

```
# Closing operation to fill black holes in the foreground/chocolates.
img_ac = area_closing(binary_img, area_threshold=10000)
```

Figure 3: Closing

## Opening

The only thing interfering with the actual chocolates and the background now, are some white spots here and there. From the original image it looks like this are crumbs and dust, e.g from the chocolates that are broken in half. To get rid of these spots I have applied the opening operation. This operation fills in the white spots, aka removes them, so they are not classified as chocolate during further analysis. The result can be seen in image (5) in Figure 5.

```
# Opening operation to remove white spots in the background
img_ao = area_opening(img_ac, area_threshold=1000)
```

Figure 4: Opening

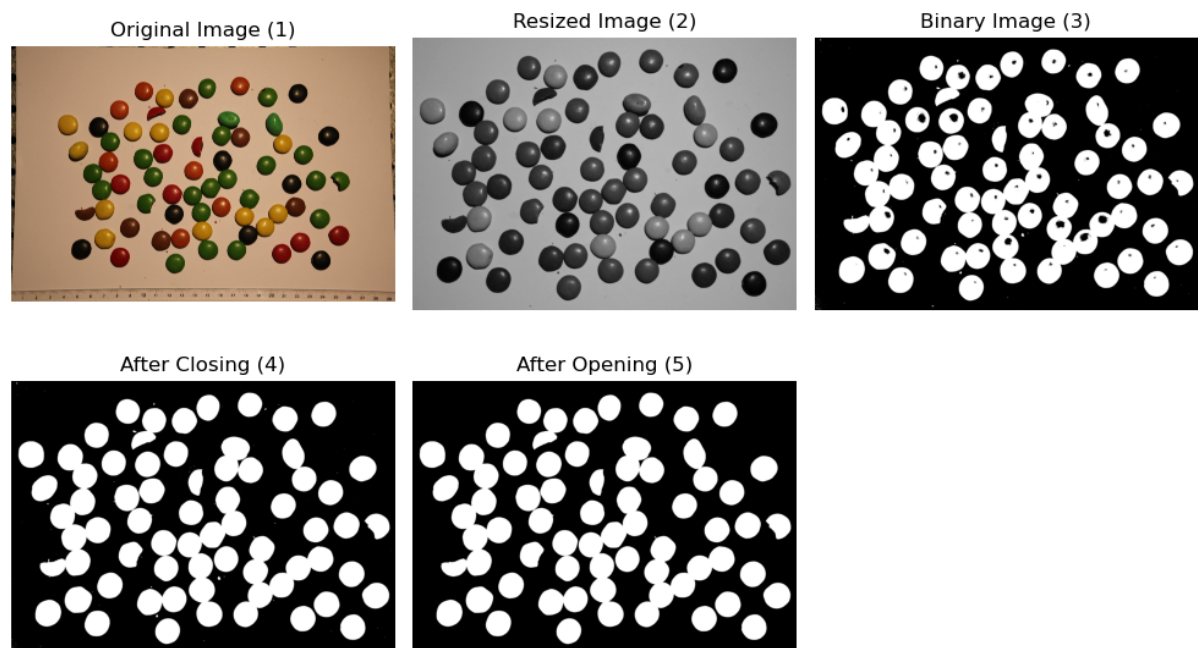


Figure 5: Resulting images from preprocessing

## Feature extraction

During feature extraction, I first computed a distance map. Then I identified local maximas in the distance map, as markers. Those markers were used when I applied the watershed algorithm to segment the chocolates. The distance map and watershedding can be seen in

```
# Compute the euclidean distance transform of the binary image
distance = ndimage.distance_transform_edt(img_ao)

# Identify local maxima in the distance map
local_maxi_coords = peak_local_max(
    distance,
    footprint = np.ones((30, 30)),
    labels = img_ao,
    min_distance = 150
)

# Create a binary array to mark the local maxima positions
local_maxi = np.zeros_like(distance, dtype=bool)
local_maxi[tuple(local_maxi_coords.T)] = True

# Label the connected components in the local maxima array to use as markers
# for the watershed algorithm
markers = ndimage.label(local_maxi)[0]
```

Figure 6: Distance mapping and watershedding

Figure 8. Further on I used the major axis length property to detect the M&Ms, as seen in Figure 7, resulting in the detection of M&Ms as seen in Figure 9..

```
# Measure properties of labeled regions in 'labels'
properties = measure.regionprops(labels)

# Extract major axis lengths of each region.
major_axis_lengths = [prop.major_axis_length for prop in properties]
MnM_threshold = np.percentile(major_axis_lengths, 96)

# Load the original image and crop to same dims as the one that has been analyzed.
img_ = io.imread(filepath)
cropped_img_ = crop(img_, ((300, 300), (600, 600), (0, 0)), copy=True)

# Iterate over each regions that are in the top of major axis lengths.
for i, prop in enumerate(properties):
    if prop.major_axis_length > MnM_threshold:
        y0, x0, y1, x1 = prop.bbox
        # Drawing a box around each object that is an M&M
        for offset in range(0, 15):
            rr, cc = draw.rectangle_perimeter(
                (y0 + offset, x0 + offset),
                end=(y1 - offset, x1 - offset),
                shape=cropped_img_.shape
            )
            cropped_img_[rr, cc] = (0, 0, 255)
        # Writes "M&M" on top of the boxes that are drawn around the detected objects.
        plt.text(x0, y0, "M&M", color='black', fontsize=7, fontweight='bold',
                bbox=dict(facecolor='white', alpha=0, edgecolor='none'),
                verticalalignment='bottom')
```

Figure 7: Detecting and highlighting M&Ms

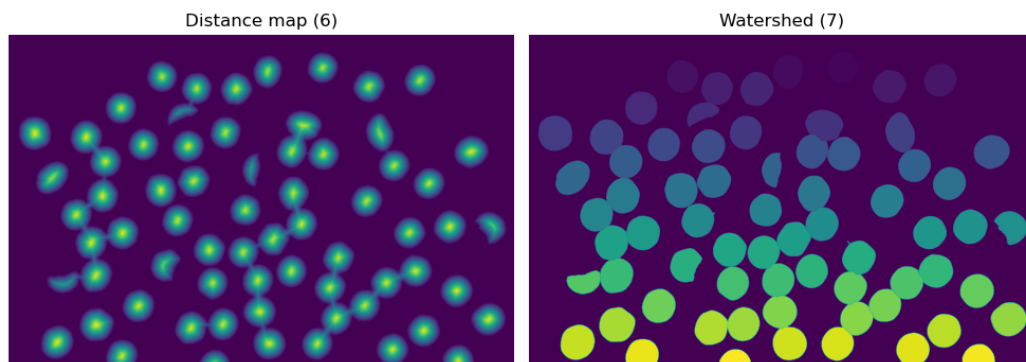


Figure 8: Distance map and watershed

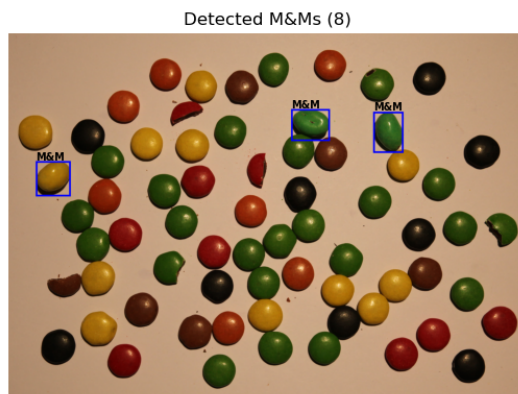


Figure 9: Result of disingushing algorithm

## Results & Analysis

My implementation of the chocolate distinguishing succeeded. During preprocessing, the only problem I came across was the threshold value computed by the `skimage` built in `threshold_otsu` method. It did not perform very well. Therefore, I experimented back and forth and found a thresholding value that performed as I wanted. I tried making the preprocessing as few steps as I could, e.g. inverting the binary image in the same step as the original image is binarized.

After having segmented the chocolates with the watershed, I stumbled my way to successfully separate the the two chocolates. To begin with I used the area property to detect only the M&Ms, but since each individual chocolate have different area independent of what type they are, it failed. Next, I tried with eccentricity, but because some of the Non-Stops were split in half, they got misclassified as M&Ms. The algorithm also failed when I used contour and perimeter, since no chocolate have 100% the same shape nor size. Lastly, I used the major axis length, in which succeeded with distinguishing M&Ms and Non-Stops.

## Conclusion

This image analysis successfully identified and distinguished the M&Ms from the Non-Stops. Using Otsu thresholding, morphological operations and watershed segmentation yielded in clear boundaries for each chocolate, allowing precise classification. By computing the major axis length, the algorithm identified potential M&Ms.

Any future steps and improvements could involve fine-tuning parameters such as the morphological thresholds and footprint to handle chocolates close to each other or various shapes of the chocolate, e.g. the Non-Stops broken in half. Implementing texture analysis could also further improve the algorithm to distinguish the chocolates due to their texture differences.